MIESC: A Multi-layer Framework for Automated Smart Contract Security Evaluation

Fernando Boiero, Sebastián Norberto Mussetta, and Norberto Gaspar Cena
Universidad Tecnológica Nacional (UTN)
Facultad Regional Villa María
Villa María, Córdoba, Argentina
{fboiero, smussetta, ngcena}@frvm.utn.edu.ar

Abstract*—Automated smart contract security tools tend to specialize: static analyzers cover known code patterns, fuzzers exercise reachable states, and auditors based on large language models (LLMs) reason about intent but vary across runs. We study whether reproducibly orchestrating them improves triage. MIESC combines 35 analysis modules across 9 layers, normalizes their outputs, filters false positives with retrieval-augmented generation (RAG) support, and records the artifacts needed to reproduce each claim. On SmartBugs-curated (143 contracts), its primary profile reaches 95.8% recall (137/143) without any LLM, and 97.9% in a Linux/Docker revalidation. A controlled, LLM-free ablation shows that MIESC's own detectors and intelligence engine add 22 contracts over its static layer and lose none (82.5% to 97.9%; exact McNemar $p < 10^{-6}$). On DeFiHackLabs reproductions of 11 real DeFi exploits ($1.59B in losses), MIESC flags the recorded vulnerability class in 9 (95% CI [52%, 95%]), a preliminary case study on attack code rather than on the victim contracts. On EVMBench business-logic vulnerabilities, scores depend on how findings are matched: a lenient keyword matcher credits the union of four provider runs with 92.5% (85% on the half that a contract-agnostic null control cannot match), while one Claude run scores 88.3% under that matcher and 27.4% under EVMBench's official judge with GPT-5; we report both as optimistic and strict estimates. Precision is low (22.2%), so MIESC is positioned as a reproducible pre-audit triage aid, not a replacement for expert review. Scripts, artifacts and a per-claim matrix are released; MIESC is available under AGPL-3.0.

***Index Terms*—smart contracts, blockchain security, vulnerability detection, defense-in-depth, multi-agent systems, retrieval-augmented generation, static analysis, formal verification**

I. Introduction

Smart contracts hold large amounts of value in decentralized finance (DeFi) protocols, and attacks on them are well documented [1]. Incidents such as the Wormhole bridge exploit ($320M, 2022) and the Euler Finance hack ($197M, 2023) show that vulnerabilities continue to cause large losses and underscore the need for rigorous analysis before deployment.

Existing automated tools—Slither [2], Mythril [3], Echidna [4]—each implement a single analysis technique and capture only a subset of vulnerabilities. Durieux et al. [5] evaluated 9 tools on 47,587 contracts and found that no single tool achieved both high precision and high recall. In practice, a security researcher preparing a pre-audit report runs 5–10 tools independently, normalizes their heterogeneous outputs by hand, and cross-references findings to filter duplicates—a workflow that does not scale to the thousands of contracts deployed weekly on Ethereum and its Layer-2 networks.

We address this gap with MIESC (Multi-layer Intelligent Evaluation for Smart Contracts), a framework that:

1) Orchestrates **13 external security tools** and **22 internal analysis modules** across **9 complementary defense layers**, each targeting a distinct class of vulnerabilities.
2) Normalizes heterogeneous tool outputs into a unified finding schema classified by the Smart Contract Weakness Classification (SWC) and Common Weakness Enumeration (CWE).
3) Filters false positives with a **rule-based filter with RAG validation**, using per-detector calibration and 93 versioned vulnerability and security-source documents.
4) Maps findings to **12 security standards and taxonomies** (e.g., ISO 27001, NIST CSF, OWASP, CWE, SWC, MITRE ATT&CK).
5) Provides an **extensible plugin system** for community-driven detector development.
6) Implements an **agentic deep-audit loop** that, on CRITICAL findings, queries a primary code-analysis model plus a dedicated verification-role model and applies bounded confidence adjustments ($+0.20$ on agreement, -0.30 on joint rejection, -0.10 + manual-review flag on disagreement), so that the LLM acts as a verifier rather than a single-shot labeler.
7) **Generates formal specifications** (Certora CVL rules, Scribble annotations, SMTChecker assertions) from findings, with a unified `miesc verify` command that runs them on the Certora Prover, Halmos, or solc's SMTChecker.
8) Extends coverage to the **Starknet / Cairo ecosystem** with 13 vulnerability classes informed by 2024–2026 exploits.
9) Introduces a **null-control protocol for LLM-auditing benchmarks**: contract-agnostic findings that measure how much credit a matching rule grants without any detection.

The evaluation measures end-to-end detection, separates the contribution of the static tools, MIESC's own detectors and its intelligence engine in a controlled ablation, measures the false-positive filter, and applies contribution 9 to EVMBench; contributions 4–8 are implemented but not separately benchmarked. Unlike prior single-technique analyzers and bench-

marking frameworks, MIESC unifies multi-tool static orchestration, RAG-backed false-positive filtering, and optional LLM layers (local or frontier) under one normalized finding schema with reproducible per-claim evidence. MIESC is designed as a *pre-audit triage tool*, not a replacement for manual expert review. By automating the multi-tool orchestration and normalization workflow, it is intended to shorten audit preparation and to enable continuous security monitoring in CI/CD pipelines, and its local-first profile keeps source code off third-party providers, which matters in regulated settings.

Our evaluation yields three main findings:

- **MIESC adds recall beyond its static layer, without any LLM.** The static and intelligence layers detect the labeled vulnerability in 137 of 143 SmartBugs-curated contracts (95.8%) on macOS and in 140 (97.9%) in a Linux/Docker revalidation. In a controlled ablation, the static layer (Slither plus a built-in pattern analyzer) reaches 82.5%; MIESC's own detectors raise this to 93.0% and its intelligence engine to 97.9%, each step gaining contracts without losing any (exact McNemar $p < 0.001$ and $p = 0.016$). The price is lower precision, which we report alongside.
- **EVMBench scores depend more on the matching rule than on the detector.** Four generic findings that mention no audited contract are credited with 50% of EVMBench's vulnerabilities by a keyword matcher, and one and the same Claude run scores 106/120 (88.3%) under the keyword matcher but 32/117 (27.4%) under EVMBench's official judge prompt with its GPT-5 judge. We release the null control so that others can check their own matchers.
- **Single-pass detection is credited with business-logic issues by a lenient matcher but rarely pins them down.** On the vulnerabilities the null control cannot match, the lenient matcher credits single LLM providers with 45–70% and their union with 85%, against 7% for static analysis alone; these optimistic estimates favor larger finding sets, and the official judge confirms 27.4% of the vulnerabilities in the same run. Agentic, hypothesis-driven auditing is the natural next step.

II. Related Work

A. Single-Technique Analysis Tools

Static analysis. Slither [2] uses intermediate representation (SlithIR) to detect vulnerability patterns with low false positive rates. Aderyn [6] implements Rust-based AST analysis for high performance. Solhint provides Solidity-specific linting rules.

Dynamic testing. Echidna [4] performs property-based fuzzing using coverage-guided input generation. Foundry's forge tool combines unit testing with fuzz-based invariant checking. Medusa extends fuzzing with corpus-based strategies.

Symbolic execution. Mythril [3] explores execution paths using SMT solving to find reachable vulnerabilities. Halmos [7] enables symbolic testing within the Foundry framework. Manticore [8] combines symbolic execution with concrete execution.

Formal verification. Certora Prover verifies user-specified properties using SMT-based techniques. SMTChecker is Solidity's built-in formal verification engine.

B. Multi-Tool Frameworks

SmartBugs [9] introduced a Docker-based framework for running smart-contract analyzers on common datasets and serves primarily as an academic benchmarking framework. SmartBugs 2.0 [10] extends it to a uniform interface over 19 tools, with parallel execution and output in the Static Analysis Results Interchange Format (SARIF).

MIESC differs from SmartBugs in three aspects relevant to this evaluation: (1) it integrates 35 analysis modules (13 external + 22 internal) across 9 layers, versus 19 tools, (2) it adds AI/LLM semantic analysis and RAG-enhanced false-positive filtering, and (3) it emits operational artifacts such as SARIF, reports, and GitHub Action outputs for CI/CD integration.

C. AI-Enhanced Security Analysis

GPTScan [11] combines GPT-4 with static analysis for logic vulnerability detection, achieving results that complement pattern-based tools. Other recent approaches apply local LLMs to multi-agent auditing, retrieval-augmented code analysis, and multi-LLM ensembles. EVMBench [12], curated by OpenAI and Paradigm from 40 Code4rena-derived repositories with 117 curated vulnerabilities, is a recent benchmark for LLM-based auditing. These tools demonstrate the potential of AI-assisted auditing but operate as standalone analyzers. MIESC combines multi-tool static orchestration *with* a multi-provider LLM ensemble, using their outputs alongside static, dynamic, and symbolic results for cross-validated findings.

III. Architecture

MIESC implements a *defense-in-depth* architecture organized into 9 layers, following the principle that multiple independent security mechanisms provide stronger protection than any single mechanism [13]. Figure 1 illustrates the analysis pipeline; each layer targets complementary vulnerability classes (Table I).

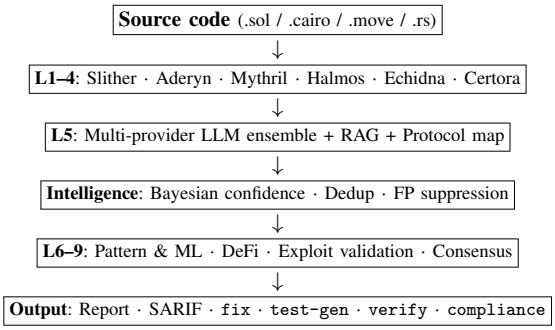


Fig. 1. MIESC pipeline: 9 defense layers with multi-provider LLM ensemble and intelligence engine for cross-tool confidence scoring.

TABLE I
MIESC'S 9 DEFENSE LAYERS

Layer	Technique	Ext.	Int.
1	Static Analysis	4	–
2	Dynamic Testing	3	–
3	Symbolic Execution	3	–
4	Formal Verification	3	1
5	AI/LLM Analysis	–	6
6	Pattern & ML Detection	–	5
7	Specialized (DeFi) Analysis	–	4
8	Exploit Validation	–	3
9	Consensus & Reporting	–	3
	Total	**13**	**22**

Ext. = third-party tools invoked as separate processes. Int. = built-in analysis modules (LLM, pattern matching, consensus).

A. Orchestration Pipeline

The analysis pipeline proceeds as follows:

1) **Input**: Solidity source code (or Vyper, Rust, Move for non-EVM chains).
2) **Tool Execution**: Each layer runs its tools in parallel with configurable timeouts. An adapter pattern normalizes tool-specific outputs into a unified `Finding` schema.
3) **Aggregation**: The Finding Aggregator deduplicates results using SWC/CWE classification and source location matching.
4) **False-positive filter**: A rule-based filter applies per-detector calibration rates, context analysis (guards, CEI patterns, OpenZeppelin usage), and Solidity version awareness.
5) **RAG Enhancement**: A Retrieval-Augmented Generation system (Section III-B) enriches findings with vulnerability context from a curated knowledge base.
6) **Report Generation**: Findings are output as JSON, SARIF (for GitHub Security integration), PDF (professional audit reports), or Markdown.

Normalized finding example. Listing 1 shows how a reentrancy report on SmartBugs' `simple_dao.sol` is represented after normalization. Every tool's output is mapped to the same fields, so deduplication, confidence scoring, SARIF export, and the remediation commands consume one schema regardless of which analyzer produced the finding.

Listing 1. Normalized MIESC finding (simplified; confidence value illustrative)

```
{"id": "slither-reentrancy-eth-1",
 "type": "reentrancy",
 "severity": "HIGH",
 "title": "Reentrancy in SimpleDAO.withdraw",
 "file_path": "reentrancy/simple_dao.sol",
 "line_start": 19,
 "tool": "slither", "layer": 1,
 "swc": "SWC-107", "cwe": "CWE-841",
 "confidence": 0.9,
 "remediation": "Update state before the external
      call or add a reentrancy guard"}
```

B. RAG System

The RAG component indexes 93 versioned documents (standards, official documentation, benchmark evidence, incident reports, audit guides and legacy taxonomies) in a ChromaDB store with all-MiniLM-L6-v2 embeddings. Entries cover reentrancy, MEV, governance, proxy, bridge, ZK/NFT/DeFi and 33 SWC patterns, and include severity guidance, exploit references, structured attack steps, detection heuristics and fix templates; the attack steps and heuristics are passed to the LLM as an actionable procedure. The knowledge base is versioned, and the version is part of every LLM cache key, so expanded evidence cannot silently reuse stale responses.

C. False Positive Filter

The FP filter employs a multi-signal approach:

- **Per-detector FP rates**: Empirically calibrated rates for 17+ detectors (e.g., Slither reentrancy: 15%, Mythril integer overflow: 8%).
- **Context analysis**: Detection of reentrancy guards, Checks-Effects-Interactions patterns, and OpenZeppelin/-Solmate library usage.
- **Solidity version awareness**: Suppression of integer overflow warnings for Solidity $\geq$ 0.8.0 (built-in overflow protection).
- **RAG validation**: Cross-reference with curated vulnerability patterns for relevance scoring.

D. Agentic Deep Audit

Beyond batch orchestration, a *DeepAuditAgent* runs a four-phase loop inspired by intelligent-agent architectures [14], with agents coordinated over a Model Context Protocol (MCP) bus [15]: (1) reconnaissance (call graph, taint analysis, risk profile), (2) targeted scanning chosen by that profile, (3) type-specific investigation of each HIGH/CRITICAL finding, and (4) synthesis of cross-finding exploit chains. For every CRITICAL finding, phase 3 asks a primary model and a separate verifier model; agreement raises confidence by 0.20, joint rejection lowers it by 0.30 and downgrades the finding to HIGH, and disagreement lowers it by 0.10 and flags the finding for manual review. When both roles map to the same model, the agent records a single opinion and leaves confidence unchanged. The agent's effect on triage effort is not measured in this paper.

E. Formal Verification Bridge

`miesc specs` turns findings into Certora CVL rules, Scribble annotations and SMTChecker assertions, and `miesc verify` runs them on whichever prover is installed (Certora, Halmos or solc's SMTChecker), normalizing the results. For access-control findings, the agent above attaches a CVL rule for the affected function, turning an informal finding into a formal obligation in one step.

F. Multi-Chain Support

A `miesc analyze` dispatcher routes each file to a chain-specific analyzer by extension. Beyond the EVM pipeline evaluated here, MIESC ships a Starknet/Cairo analyzer with 13 vulnerability types drawn from 2024–2026 incidents (e.g., zkLend's stale-price read and Braavos account re-initialization)

and scaffolded Move and Solana analyzers; none of these is evaluated in this paper.

IV. Evaluation

A. Experimental Setup

We evaluate MIESC on three complementary datasets. First, SmartBugs-curated [5], [9] provides 143 Solidity contracts across 10 vulnerability categories and serves as the classical pattern-based benchmark. Second, DeFiHackLabs reproductions of 11 confirmed DeFi incidents serve as a preliminary case study of whether MIESC flags the vulnerability class behind each attack. Third, EVMBench [12] evaluates semantic business-logic vulnerabilities from Code4rena-derived audits.

Environment: macOS ARM64 (Apple Silicon), 48 GB unified memory. The SmartBugs artifacts reported here were produced with MIESC v5.4.0/v5.4.1 and Python 3.14.4 on 2026-05-06 through 2026-05-09, then consolidated in this paper's v2 evidence freeze on 2026-06-23. A later revalidation of the same release (v5.4.1) in a Linux Docker container, and the ablation of Section IV-B2, use Linux builds of the compilers. EVMBench provider runs use stored MIESC result artifacts; the reproducible ensemble artifact is generated by `generate_paper1_artifacts.py`, which computes a deterministic union over matched `audit:vulnerability` keys.

Evaluation profiles: The full-corpus SmartBugs result uses layers 1, 6, and 7 because these layers cover the static analyzers, SmartBugs-specific detectors, ML classifiers, and specialized checks that dominate this benchmark while keeping the run deterministic and fast. To verify that the complete toolchain remains executable, we also run a separate full-tool smoke test over layers 1–9 on the first SmartBugs contract and store its JSON/JSONL artifacts. We treat this smoke run as integration evidence, not as a corpus-level accuracy claim.

B. Results

TABLE II
Results on SmartBugs-curated (143 contracts)

Configuration	Precision	Recall	F1
MIESC (layers 1, 6, 7)	22.2%	95.8%	36.0%

1) Overall Performance: Table II shows the latest full-corpus reproducible SmartBugs profile stored in the repository. Following the evaluation script, a true positive (TP) is a contract whose labeled vulnerability class is reported, a false positive (FP) is a vulnerability class reported for a contract that is not labeled with it, and precision is TP/(TP+FP); recall and F1 follow from these counts (137 TP, 481 FP, 6 misses). MIESC detects the labeled vulnerability class in 137 of 143 contracts (95.8% recall, 36.0% F1). For context only, Durieux et al. [5] reported on an earlier, 69-contract version of this annotated corpus that nine tools combined detected 42% of the vulnerabilities and the most accurate single tool (Mythril) 27%. Corpus version and counting unit differ, so these figures are not directly comparable to ours. This benchmark is useful for classical vulnerability patterns, but it is not the central evidence for business-logic vulnerabilities; that role is played by EVMBench (Section IV-B6).

The low precision (22.2%) comes mostly from category-level alerts on legacy 0.4.x code: arithmetic (128 of the 481 false-positive pairs), reentrancy (109) and unchecked low-level calls (64) are reported for many contracts not labeled with them. For pre-audit triage, recall is the priority: a missed vulnerability can reach production, whereas a false positive costs reviewer time.

2) Ablation: What MIESC Adds Beyond the Tools: To separate MIESC's contribution from that of its static layer, we re-ran the corpus three times with the same release (v5.4.1) in one Linux environment, enabling one component at a time (Table III): `miesc evaluate corpus` with `--layers 1 --no-intelligence`, with `--layers 1,6,7 --no-intelligence`, and with `--layers 1,6,7`. The static layer here consisted of Slither 0.11.6 and MIESC's built-in Code4rena-style pattern analyzer; Aderyn, Solhint, Wake and Semgrep were not installed. No LLM is involved, so the runs are deterministic.

TABLE III
Controlled Ablation on SmartBugs-curated (143 contracts, no LLM)

Configuration	Recall	95% CI	Prec.
Static layer 1	118/143 (82.5%)	[75.5, 87.9]	29.3%
+ MIESC detectors (layers 6, 7)	133/143 (93.0%)	[87.6, 96.2]	21.1%
+ Intelligence engine	140/143 (97.9%)	[94.0, 99.3]	19.8%

Exact McNemar tests on paired contracts: layers 6–7 gain 15 contracts and lose none ($p = 6.1 \times 10^{-5}$); the intelligence engine gains 7 and loses none ($p = 0.016$). The three outputs and a summary script are released.

Each step adds detected contracts without losing any. MIESC's own detectors recover access-control (6), front-running (4), denial-of-service (3), short-address (1) and reentrancy (1) cases that the static layer misses, and the intelligence engine adds six further access-control contracts and one reentrancy contract. Recall rises at the cost of precision (29.3% to 19.8%; F1 falls from 43.3% to 32.9%), measured against a reduced static layer (Slither plus the built-in pattern analyzer), which is the expected trade-off for a triage tool and the reason the false-positive filter matters (Section V-C). In this environment the full profile detects 140 of 143 contracts, the same result as a Linux Docker revalidation of the release (140/143, missing only the three "other" contracts) and three more than the canonical macOS run (137/143), which additionally missed two front-running contracts and the short-address contract. We have not traced that platform difference to a specific cause, so we keep 95.8% as the primary, conservative figure and compare configurations only within one environment.

3) Category-Specific Detection: Table IV breaks recall down by category. Seven of ten categories reach 100% recall: reentrancy, unchecked low-level calls, arithmetic, time manipulation, access control, bad randomness, and denial of service. For arithmetic (reported on all 143 contracts) and reentrancy (140) this recall carries little information, which is why we report precision alongside it. Front running reaches 50% (2 of

TABLE IV
PERFORMANCE BY VULNERABILITY CATEGORY (SMARTBUGS-CURATED)

Category	Precision	Recall	F1
Reentrancy	22.1%	**100%**	36.3%
Unchecked Low-Level Calls	44.8%	**100%**	61.9%
Arithmetic	10.5%	**100%**	19.0%
Time Manipulation	12.5%	**100%**	22.2%
Access Control	27.7%	**100%**	43.4%
Bad Randomness	14.8%	**100%**	25.8%
Denial of Service	11.5%	**100%**	20.7%
Front Running	28.6%	50%	36.4%
Short Addresses	0%	0%	0%
Other	0%	0%	0%

Latest full-corpus local JSON: `paper1_smartbugs_eval_layers_1_6_7.json`; layers 1, 6, and 7; 143 contracts. Tool-level failures on legacy syntax are counted as per-contract misses when no other layer detects the category.

4 contracts) and short addresses 0% (1 contract) in this macOS run; the Linux revalidation of the same release detects all five, although there its short-address and front-running detectors fire on 43 and 47 contracts, respectively. The only misses common to both platforms are the three contracts of the heterogeneous "other" class, which have no single vulnerability pattern.

4) Real-World Exploits: A Preliminary Case Study: As a preliminary case study beyond academic benchmarks, we ran MIESC on DeFiHackLabs reproductions of 11 confirmed DeFi exploits with $1.59 billion in combined losses, drawn from a 25-incident curated corpus (Table V); the 11 are the incidents with an available reproduction. Each reproduction is an attacker-side Foundry harness: it declares interfaces to the victim contracts and encodes the attack sequence, but does not contain the victim's source code. The run used MIESC's DeepAuditAgent (v5.1.6) without LLM layers; per-incident results are transcribed from that run's report, and we checked whether it flagged the vulnerability class recorded for each incident.

TABLE V
VULNERABILITY CLASS FLAGGED IN DEFIHACKLABS REPRODUCTIONS OF REAL-WORLD EXPLOITS (11 INCIDENTS, USD 1.59B COMBINED LOSSES)

Vulnerability	Exploits	Flagged	Rate
Reentrancy	3	3	**100%**
Access Control	3	3	**100%**
Flash Loan	2	2	**100%**
Arithmetic	1	1	**100%**
Oracle Manipulation	1	0	0%
Governance Attack	1	0	0%
Overall	**11**	**9**	**81.8%**

MIESC flagged the recorded vulnerability class in 9 of 11 reproductions (81.8%; 95% CI [52%, 95%] by the Wilson method). Two caveats limit what this shows. First, with $n = 11$ the interval spans from about one half to almost all incidents. Second, because the analyzed code is the attack harness rather than the victim contract, a hit shows that MIESC recognizes the attack pattern as written in the reproduction, not that it would have found the flaw in the deployed victim code. We therefore present this as a case study of pattern recognition on attack code, not as a recall estimate for vulnerable contracts; re-running it on verified victim sources is planned. Two labels need caution. Ronin's root cause was compromised validator keys rather than a code defect, and Euler is labeled reentrancy in our corpus although its documented mechanism is a donation-based manipulation. The Parity loss ($280M) was a freeze caused by an accidental self-destruct rather than a theft. Excluding Ronin and Euler, MIESC flags 7 of 9 incidents (78%; 95% CI [45%, 94%]).

5) Execution Performance and Full-Tool Smoke Test: The reproducible profile analyzed all 143 contracts in 737 s (5.15 s per contract, 11.6 contracts per minute) with 100% evaluator completion. To avoid conflating this fast profile with full-tool execution, we also ran all 9 layers (180 s per-tool timeout) on the first SmartBugs contract: the run took 183.7 s, and its trace records every layer plus the intelligence stage, with unavailable tools logged explicitly (e.g., Certora without a license, Vertigo on non-Truffle/Hardhat inputs). This smoke run supports the integration claim only; all corpus-level metrics come from the reproducible 143-contract profile.

6) EVMBench: Business-Logic Vulnerabilities (Exploratory): To evaluate MIESC on vulnerabilities requiring semantic understanding—business logic flaws, economic exploits, and protocol-level errors—we benchmark against an EVMBench local high-severity extraction. The official EVMBench paper describes 117 curated vulnerabilities; our local artifact contains 120 high-severity audit findings across 40 audit entries. We report this distinction explicitly and publish the mapping artifact for reproducibility. Unlike SmartBugs' synthetic patterns, EVMBench vulnerabilities require reasoning about protocol invariants, value flows, and multi-contract interactions.

MIESC supports multiple analysis modes through a unified pipeline: the same Chain-of-Thought audit prompt, protocol-specific RAG enrichment, and finding normalization are applied regardless of provider, enabling fair comparison.

Source preparation. For each audit, MIESC selects implementation files by size (excluding tests, mocks, and interfaces), concatenates them into a single context (100KB budget in the recorded EVMBench runs), and prepends protocol context summarizing contract hierarchy, external calls, and extracted invariants. If a provider fails for an audit, the failure is retained in the provider artifact rather than silently removed.

Matching protocols. We score findings under two protocols. Our original *lenient matcher* credits a ground-truth vulnerability as detected when any finding for that audit triggers any one of several signals: an overlapping function name, a shared keyword from one of 10 broad vulnerability categories, two shared words, a shared bigram, or shared word stems; an optional LLM judge can only add matches. The *official grader* applies EVMBench's detect-mode judge prompt, copied verbatim, to a report built from the findings: a strict LLM judge that requires the same mechanism at the same code location. We run it with GPT-5, EVMBench's judge model, and with GPT-4o for comparison. Because the stored provider runs do not retain the full finding text, we made a

dedicated run (Claude Sonnet 4.6, single pass, the six largest in-scope files of each audit; 796 findings) and scored that same set of findings under every protocol.

Matcher validation. We tested whether the lenient matcher's credits reflect detection. As a null control, we fed it four generic findings (missing access control, reentrancy, oracle manipulation, incorrect accounting) that reference no audited contract. The matcher credits these contract-agnostic findings with 60 of the 120 vulnerabilities (50.0%). The four provider runs report 218 to 434 findings over the 40 audits, so credit on those 60 vulnerabilities cannot be attributed to detection. We therefore also score every run on the other 60, the *control-resistant* subset, where the generic findings earn nothing. Table VI sets both views beside the official grader. In the dedicated run, 30 of the 32 vulnerabilities that the GPT-5 judge accepts are also credited by the lenient matcher, so the official detections are almost a subset of the lenient credits; the converse does not hold, since GPT-5 confirms only 30 of the 106 lenient credits. The two judge models agree on 104 of 117 vulnerabilities (GPT-5 alone accepts 10, GPT-4o alone 3; exact McNemar $p = 0.09$).

TABLE VI
EVMBENCH CREDITED RECALL BY MATCHING PROTOCOL

Findings scored	Protocol	All vulns.	Control-resist.
4 generic findings (null control)	Lenient	60/120 (50.0%)	0/60 (0%)
Static-only run	Lenient	22/120 (18.3%)	4/60 (7%)
Claude Sonnet 4.6 run	Lenient	99/120 (82.5%)	42/60 (70%)
Union of 4 provider runs[a]	Lenient	111/120 (92.5%)	51/60 (85%)
Dedicated Claude run[c]	Lenient	106/120 (88.3%)	47/60 (78%)
Dedicated Claude run[c]	Official, GPT-4o[b]	25/117 (21.4%)	—
Dedicated Claude run[c]	**Official, GPT-5[b]**	**32/117 (27.4%)**	—

[a]Claude Sonnet 4.6, GPT-4o, GPT-5 and local Qwen2.5-Coder 32B. [b]EVMBench's detect-mode judge prompt applied to a report of the run's findings; GPT-5 is EVMBench's judge model. 95% Wilson CIs: GPT-5 [20, 36], GPT-4o [15, 30]. Control-resistant credit per single provider: Claude 70%, GPT-5 63%, GPT-4o 55%, Qwen 45%; 95% Wilson CIs: Claude [57, 80], union [74, 92]. [c]One Claude Sonnet 4.6 run (2026-10-16) whose findings are scored under every protocol; 8 of 197 per-file model responses could not be parsed and contributed no findings. An earlier run of the same pipeline scored 12/117 with the GPT-4o judge, an indication of run-to-run variance. The official corpus defines 117 vulnerabilities; our local extraction lists 120. The null-control script, the grader reimplementation, and all provider artifacts are released.

Result. On the control-resistant subset the lenient matcher discriminates between runs: static-only analysis is credited with 4 of 60 (7%), the Claude run with 42 (70%), and the four-provider union with 51 (85%). These figures remain optimistic, because larger finding sets offer more keyword surface to a lenient matcher. Under the official judge prompt, which requires the exact mechanism and location, the same Claude findings that the lenient matcher credits with 88.3% recover 32 of 117 business-logic vulnerabilities with the GPT-5 judge (27.4%; 95% CI [20%, 36%]) and 25 with GPT-4o (21.4%). We read the two protocols as optimistic and strict estimates of the same run. Because the findings are identical, the gap between them is due to the matching protocol alone. The low strict score itself we attribute partly to task design, since EVMBench targets multi-turn, hypothesis-driven auditing whereas MIESC performs single-pass detection on at most six files per audit, an explanation we have not tested. The broader lesson is methodological. The same findings move from 27.4% to 88.3% depending on how they are matched, and contract-agnostic text reaches 50% under a keyword-based matcher, so the protocol and judge must be reported with every EVMBench score. The strict figure uses EVMBench's judge prompt and judge model, but it scores a single-pass detection report rather than an agent working in EVMBench's own harness, so it is not a leaderboard entry; we recommend that future work report a null control alongside any custom matcher.

V. DISCUSSION

A. Layer Contribution Analysis

The main pattern is a split between classical vulnerability signatures and semantic reasoning. On SmartBugs-curated, MIESC's static+intelligence profile (layers 1, 6, 7 plus the pattern-based intelligence engine) reaches 95.8% recall (137/143) without invoking any LLM, and the ablation (Section IV-B2) attributes 22 of the detected contracts to MIESC's own detectors and intelligence engine rather than to the wrapped analyzers. The 6 remaining macOS misses—3 "other" cases, 2 front-running cases, and 1 short-addresses case—are not uniform failures: on Linux, the same release detects the front-running and short-address cases without any LLM (140/143). The 3 "other" contracts remain unresolved on both platforms because the category does not define a single vulnerability pattern.

On EVMBench, the lenient matcher credits single LLM providers with 45–70% on the control-resistant subset, and their union with 85%, against 7% for static analysis alone, an optimistic estimate; under the official judge prompt with GPT-5, however, the same Claude findings reach only 27.4%, so the semantic layers do not yet pin down the exact mechanism of deep, protocol-specific flaws. The evidence supports a narrower claim than "more tools is better": each layer is useful only when matched to the vulnerability class it can actually observe.

B. Comparison with SmartBugs 2.0

SmartBugs 2.0 [10] is the closest comparable framework, integrating 19 analysis tools with Docker-based execution. However, SmartBugs focuses on providing a *benchmarking platform* and does not publish detection metrics (precision, recall) on its own curated dataset. The tools it integrates (Mythril, Slither, Securify, Oyente, etc.) were benchmarked by Durieux et al. [5] on an earlier, 69-contract version of the annotated corpus.

A direct recall comparison is not possible because SmartBugs does not publish framework-level detection metrics on the curated dataset. On that earlier version, the nine tools combined detected 42% of the vulnerabilities [5]; MIESC's profile reaches 95.8% contract-level recall on the current 143-contract version. Because corpus version and counting unit differ, we read this as an indication rather than a controlled comparison. Our own ablation provides the controlled comparison that the published figures cannot: under one protocol, the

static layer reaches 82.5% and the full MIESC profile 97.9% (Section IV-B2). The differentiator is therefore not tool count alone but MIESC's own detectors and its intelligence engine, together with normalization across heterogeneous tool outputs and false-positive filtering.

C. False Positive Management

Contract-level precision (22.2%) remains a usability concern. The latest SmartBugs-curated run reports 481 false-positive contract–category pairs for 143 contracts, dominated by arithmetic, reentrancy and unchecked-call alerts on legacy code. The FP filter addresses this through four mechanisms. The first is per-detector calibration based on empirical FP rates from our benchmark runs (e.g., Slither's `solc-version` detector has near-100% FP rate on modern contracts). The second is Solidity version awareness, which suppresses integer overflow warnings for contracts using $\geq$0.8 (which has built-in overflow protection). The third is library detection, which recognizes OpenZeppelin and Solmate safe patterns. The fourth is a set of context-aware filters that suppress patterns in benign contexts (e.g., `block.timestamp` used only in timelock `require()`, block variables not used for entropy generation, pre-0.8 contracts without actual financial arithmetic). We measured the production filter separately, in a finding-level evaluation over the same detectors and the corpus's 232 labeled vulnerable lines: it removes 207 of 1,263 false positives (16%) without losing any true positive, raising finding-level precision from 13.3% to 15.5%. The filter is therefore safe but conservative; larger precision gains will require a learned classifier trained on auditor-labeled findings. Since v5.2.0, the intelligence engine performs semantic deduplication, context-aware FP suppression, and cross-tool Bayesian confidence scoring.

D. Analysis of Missed Exploits

Two reproductions were not flagged: the BonqDAO oracle manipulation ($120M) and the Beanstalk governance flash loan attack ($182M). Both require *semantic understanding* of protocol-specific logic that static pattern matching cannot capture. BonqDAO's vulnerability was in the interaction between a Tellor oracle and a custom stablecoin—detecting this requires modeling the oracle's trust assumptions, not just code patterns. Beanstalk's attack exploited governance voting with flash-borrowed tokens in a single transaction—a logical flaw invisible to pattern-based analysis. These cases motivate protocol-aware semantic analysis in Layers 5–7.

E. Local and Open-Weight Models

MIESC's static and intelligence layers run without any LLM, and the LLM layers accept local or open-weight models as well as frontier APIs. This matters for reproducibility as much as for cost or privacy: a versioned open-weight model executed under the analyst's control can be frozen with the other artifacts, whereas a proprietary API may change between runs. Because our provider comparison relied on the lenient matcher, we do not rank providers here; a provider comparison under the official grader is future work.

F. Deployment Profiles and CI/CD Trade-offs

The two recorded execution profiles differ by more than an order of magnitude in latency: the reproducible static-focused profile averages 5.15 s per contract (Section IV-B5), whereas the full 9-layer run took 183.7 s on a single contract; the latency of local-model LLM layers was not measured separately. This suggests a two-tier deployment. On every commit or pull request, teams can run the static and intelligence layers with a local model (`miesc scan --model ollama`), which keeps source code private and incurs no API cost. Before a mainnet deployment or an external audit, they can run the full multi-provider ensemble and the formal-verification commands, accepting higher latency, cost, and provider exposure. SARIF export lets findings from either tier flow into existing issue trackers, so human reviewers start from a normalized list rather than from raw tool output.

G. Security of the Framework Itself

A security tool is itself an attack target, and we have not performed a formal security evaluation of MIESC. We outline the threat model that such an evaluation should cover. The analyzed contract is untrusted input, which opens four attack surfaces: (1) *command injection* through file names or paths passed to external tools; (2) *prompt injection* through comments or identifiers in the contract that try to steer the LLM layers, for example toward dismissing a finding; (3) *knowledge-base poisoning* if untrusted documents enter the RAG corpus; and (4) *artifact tampering*, for example by a compromised CI/CD runner, that alters stored results or the claims matrix after a run. Two further surfaces belong to the deployment rather than the input: the REST API and MCP server, which should not be exposed to untrusted networks, and the LLM provider API keys, which are read from the environment and must be kept out of logs and stored artifacts. Current mitigations are partial. External tools are invoked as argument lists rather than through a shell; regular expressions applied to repository content are bounded to prevent injection-driven ReDoS; the RAG corpus is versioned and its version is part of the LLM cache key; and the multi-LLM consensus flags disagreements for manual review instead of silently accepting one model's verdict. Systematic adversarial testing of the LLM layers against prompt injection, and signed provenance for stored artifacts, are future work.

H. Threats to Validity

Internal validity. The ablation was run in a different environment from the canonical profile (Linux, without Aderyn, Solhint, Wake or Semgrep), so its absolute figures differ slightly; we compare configurations only within that environment. The SmartBugs-curated ground truth was established by Durieux et al. [5] and may contain labeling errors. Our finding classifier maps MIESC tool outputs to SmartBugs categories using SWC IDs and keyword matching, which may introduce classification errors. EVMBench matching was the largest threat: our original multi-signal matcher credits contract-agnostic findings with 50% recall, and the same

Claude findings score 88.3% under it against 27.4% under the official judge prompt with GPT-5. We therefore pair every lenient figure with its control-resistant counterpart and the strict-judge result, treat the latter as the closest comparison with other work (it scores a detection report, not an agent in EVMBench's harness), report run-to-run variance (12/117 and 25/117 with the GPT-4o judge in two runs), and publish the null-control script, the official-grader reimplementation, and all provider artifacts.

External validity. SmartBugs-curated consists mostly of small legacy contracts (142 of 143 declare a 0.4.x compiler pragma), which may not represent modern DeFi protocols. We partly address this with the real-world exploit case study (2017–2023 incidents), though it analyzes attack reproductions and the sample is small, and with EVMBench business-logic findings from Code4rena-derived audits. The official EVMBench paper reports 117 curated vulnerabilities, while our local artifact evaluates 120 high-severity findings; this difference is explicitly documented in the reproducibility files.

Construct validity. Recall is our primary metric because missed vulnerabilities have higher cost than false positives in pre-audit triage. However, precision remains important for usability—a tool that generates too many false positives will be abandoned by practitioners.

Reproducibility. All benchmark scripts, ground truth data, and evaluation methodology are included in the repository. The EVMBench ensemble and claims matrix can be reproduced with:

```
SOURCE_DATE_EPOCH=0 python3
benchmarks/generate_paper1_artifacts.py
```

This generates `evmbench_ensemble_40.json` and `paper1_claims_matrix.json`. The EVMBench null control is reproduced with `benchmarks/evmbench_matcher_null_control.py`, and the dedicated run scored under every protocol with `benchmarks/evmbench_same_run_protocols.py`, which stores the findings and per-vulnerability verdicts. SmartBugs can be reproduced with the command listed in `PAPER1_REPRODUCIBILITY.md`; the fixed full-corpus profile is `miesc evaluate corpus ... --layers 1,6,7 --timeout 120`, and the same document records the full-tool smoke command for layers 1–9. The Slither adapter selects an installed `solc-select` artifact compatible with each contract pragma (for example, `^0.4.x` contracts use `solc-0.4.26`), avoiding the historical failure mode where `solc 0.8.20` was applied to legacy Solidity.

I. Limitations

- The Starknet/Cairo analyzer is not evaluated against a ground-truth corpus in this paper.
- Move (Sui/Aptos) and Solana/Anchor analyzers are scaffolded for community contribution but not yet evaluated against ground-truth benchmarks.
- Full 9-layer analysis is substantially slower than the reproducible full-corpus profile because it invokes LLM and formal-verification adapters; in the recorded smoke run, one contract required 183.7s.
- The toolchain is not fully portable: Certora requires a commercial license, Mythril and Manticore need isolated environments, and Echidna/Medusa are amd64-only. Unavailable tools are recorded as skipped rather than silently omitted.
- Scalability to large projects is not evaluated. SmartBugs contracts are small single files, and the EVMBench runs analyze at most a 100 KB context or six files per audit; performance on multi-file projects with deep dependency trees and contracts above that budget remains to be measured.
- LLM-based layers depend on model quality and introduce non-determinism; the multi-LLM consensus mechanism (Section III-D) mitigates but does not eliminate this; results may vary across Ollama model versions.
- The real-world exploit case study analyzes 11 attack reproductions rather than victim code; re-running it on verified victim sources and expanding it to 100+ EVM incidents is planned, and non-EVM corpora remain future work.

VI. Conclusion

This work reports a staged evaluation of MIESC rather than a single headline score. On pattern-based vulnerabilities (SmartBugs), the static+intelligence profile reaches 95.8% recall (137/143) without any LLM on macOS and **97.9%** (140/143) in a Linux/Docker revalidation, and a controlled ablation attributes 22 detected contracts to MIESC's own detectors and intelligence engine rather than to its static layer. On business-logic vulnerabilities, the result depends on the matching protocol. Under a lenient matcher the union of four provider runs is credited with 92.5%, and with 85% on the control-resistant half of EVMBench; in a single Claude run scored under every protocol, the lenient matcher credits 88.3% and EVMBench's official judge prompt with GPT-5 confirms 27.4% (32/117). Our null control shows that a keyword-based matcher credits contract-agnostic findings with 50%, so EVMBench numbers obtained with custom matchers need a control of this kind to be interpretable. This marks the boundary of the current design. MIESC is effective at orchestrating tools for classical patterns and at producing reproducible, normalized evidence, but deep business-logic auditing requires hypothesis-driven exploration of the codebase. MIESC's agentic loop only deepens the analysis of findings the pipeline has already produced; it does not yet generate and test new hypotheses.

The practical contribution is the workflow boundary. `miesc scan contract.sol --model ollama` runs Slither, Aderyn, an intelligence engine with Bayesian cross-tool confidence, protocol-specific RAG enrichment, and a local LLM audit without sending code to a cloud provider. Teams can then decide whether the remaining findings justify a frontier provider or manual review.

The agentic and formal-verification components follow the same principle: an isolated LLM verdict or a standalone prover

run is rarely sufficient, but each can add evidence when its assumptions are explicit. MIESC therefore records LLM consensus, generated specifications, and prover inputs as separate artifacts rather than collapsing them into a single confidence label. The 2024–2026 Cairo exploit coverage applies this idea to a second ecosystem by encoding lessons from zkLend and Braavos as machine-checkable patterns.

Three directions for future work emerge from these results. First, an agentic detection loop (hypothesis formation and code-path verification over multiple turns) is needed for business-logic vulnerabilities, evaluated as an agent in EVMBench's own harness. Second, precision needs improvement: the production filter removes 16% of false positives without losing true positives, and we plan to train a learned false-positive classifier once a labeled corpus of auditor-validated findings is available. Third, the exploit case study should move to verified victim sources and expand to 100+ EVM incidents to support statistical claims. The remediation commands (`miesc fix`, `test-gen`, `compliance`) are evaluated in a companion study [16]; linking the generated Foundry tests to the generated CVL rules, so that each fix is shown to discharge its rule, remains open.

MIESC is available under AGPL-3.0 (see *Data Availability*).

Acknowledgment

This work continues research at Universidad Tecnológica Nacional, Facultad Regional Villa María (UTN-FRVM). The authors thank the Ethereum security community and the developers of Slither, Mythril, Echidna, Foundry, Aderyn, and Halmos for their foundational contributions.

Data Availability

MIESC source code, benchmark scripts, ground truth datasets, and evaluation results are publicly available at https://github.com/fboiero/MIESC under AGPL-3.0. The SmartBugs-curated dataset is from Durieux et al. [5]. The exploit reproductions come from DeFiHackLabs (https://github.com/SunWeb3Sec/DeFiHackLabs) and are not bundled; `paper1_exploits_eval_20260621.json` records the per-incident results. Paper-specific reproducibility artifacts are documented in `PAPER1_REPRODUCIBILITY.md`, with generated outputs `paper1_claims_matrix.json`, `evmbench_static_40.json` and `evmbench_ensemble_40.json`; the ablation outputs and summary script (`paper1_ablation_*`), the EVMBench null control and the same-run scoring script are under `benchmarks/`, with the dedicated run's findings and verdicts in benchmarks/results/evmbench/same_run_claude_20261016/.

References

[1] N. Atzei, M. Bartoletti, and T. Cimoli, "A survey of attacks on Ethereum smart contracts (SoK)," *Proceedings of the 6th International Conference on Principles of Security and Trust (POST)*, pp. 164–186, 2017.

[2] J. Feist, G. Grieco, and A. Groce, "Slither: A static analysis framework for smart contracts," in *2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*. IEEE, 2019, pp. 8–15.

[3] B. Mueller, "Smashing Ethereum smart contracts for fun and real profit," in *HITB Security Conference*, 2018.

[4] G. Grieco, W. Song, A. Cygan, J. Feist, and A. Groce, "Echidna: Effective, usable, and fast fuzzing for smart contracts," in *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2020, pp. 557–560.

[5] T. Durieux, J. F. Ferreira, R. Abreu, and P. Cruz, "Empirical review of automated analysis tools on 47,587 Ethereum smart contracts," in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (ICSE)*. ACM, 2020, pp. 530–541.

[6] Cyfrin, "Aderyn: Rust-based Solidity ast analyzer," https://github.com/Cyfrin/aderyn, 2024.

[7] a16z crypto, "Halmos: Symbolic testing for EVM smart contracts," https://github.com/a16z/halmos, 2023.

[8] M. Mossberg, F. Manzano, E. Hennenfent, A. Groce, G. Grieco, J. Feist, T. Brunson, and A. Dinaburg, "Manticore: A user-friendly symbolic execution framework for binaries and smart contracts," in *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2019, pp. 1186–1189.

[9] J. F. Ferreira, P. Cruz, T. Durieux, and R. Abreu, "SmartBugs: A framework to analyze Solidity smart contracts," in *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. ACM, 2020, pp. 1349–1352.

[10] M. di Angelo, T. Durieux, J. F. Ferreira, and G. Salzer, "SmartBugs 2.0: An execution framework for weakness detection in Ethereum smart contracts," in *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2023, pp. 2102–2105.

[11] Y. Sun, D. Wu, Y. Xue, H. Liu, H. Wang, Z. Xu, X. Xie, and Y. Liu, "GPTScan: Detecting logic vulnerabilities in smart contracts by combining GPT with program analysis," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE)*. ACM, 2024, pp. 1–13.

[12] J. Wang, A. Bigger, X. Xu, J. W. Lin, A. Applebaum, T. Patwardhan, A. Yukseloglu, and O. Watkins, "EVMbench: Evaluating AI agents on smart contract security," *arXiv preprint arXiv:2603.04915*, 2026.

[13] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975.

[14] M. Wooldridge and N. R. Jennings, "Intelligent agents: Theory and practice," *The Knowledge Engineering Review*, vol. 10, no. 2, pp. 115–152, 1995.

[15] Anthropic, "Model context protocol specification," https://modelcontextprotocol.io, 2024.

[16] F. Boiero, S. N. Mussetta, and N. G. Cena, "From findings to verifiable remediation artifacts: A smart contract security pipeline," in *IV Simposio Internacional de Blockchain y Ciberseguridad (RIBCi–CYTED 2026)*, ser. Livros de Atas. Porto, Portugal: Edições CEOS.PP, ISCAP, 2026.